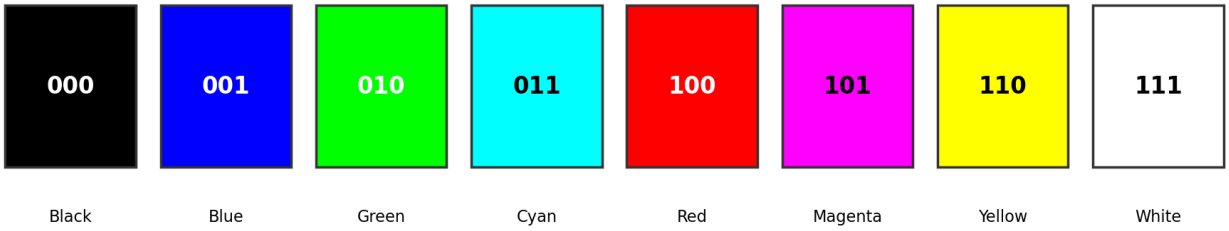
CubeOCC

FPGA Optical Camera Communication using RGB-Cube Symbols

Architecture review, optical frame protocol, simulation plan, implementation options, and one-month work split

Recommended GitHub repository: cubeocc-fpga

Natural 3-bit RGB-cube alphabet: each channel directly represents one bit



The eight display symbols are the eight corners of the RGB cube. The natural mapping makes each R/G/B channel one data bit.

Prepared for a two-person, one-month prototype with no FPGA/camera hardware available initially.

1. Executive summary

Your project is a valid and interesting Optical Camera Communication (OCC) receiver concept: an RGB LED/display grid carries data, a camera captures the grid, FPGA logic recovers geometry and symbols, ECC reconstructs the bitstream, and a testbench measures BER and throughput under labeled visual conditions.

The central design is strong, but four architectural changes will make the project much more realistic and finishable in one month:

- Do not store full RGB frames primarily in flip-flops/latches. Use streaming, line buffers, and BRAM/URAM ROI banks. Flip-flops should hold control state, descriptors, counters, small windows, and registers - not megapixel frame stores.
- Treat the two cameras as a calibrated pair. A simple constant x/y offset is generally insufficient because focal length, lens distortion, perspective, and mounting geometry differ. The tracker should produce ROI/transform parameters that are mapped into the RGB-camera coordinate system.
- Prefer a streaming HLS DATAFLOW pipeline before duplicating three complete image-processing workers. Three frame workers can be a stretch option after one worker is correct and resource use is measured.
- Keep the clock/toggle cell, but add an ECC-protected frame sequence number. The toggle cell detects duplicate captures; the sequence number detects dropped or reordered frames.

Recommended scope: Build a correct 8x8 simulation first, with fixed four-corner tracking, RGB-cube classification, START/DATA/STOP framing, one ECC mode, CSV metrics, and camera-like streaming. Add multi-ECC comparison, stronger glare handling, 10x10 grids, and multi-worker scheduling only after the baseline works.

2. Naming recommendation

OCC is already the established abbreviation for Optical Camera Communication, which matches the project very well. The problem is only that "OCC" by itself is too generic for a GitHub repository. A distinctive suffix should describe the RGB-cube/grid modulation.

Rank	Project name	Suggested repo	Why it fits
1	CubeOCC	cubeocc-fpga	Clear OCC identity + the 8 RGB cube-corner symbols. Short and memorable.
2	ChromaOCC	chromaocc-fpga	Emphasizes color-domain optical communication.
3	OCC-RGBGrid	occ-rgbgrid	Very descriptive and easy to search.
4	PrismLink FPGA	prismlink-fpga	Good product-style name; less explicit about camera reception.
5	OptiGrid RX	optigrid-rx-fpga	Highlights the grid receiver and FPGA processing.
6	CubeLink OCC	cubelink-occ	Descriptive, but "CubeLink" is a more generic term.

Recommended full title: "CubeOCC: FPGA Optical Camera Communication using RGB-Cube Grid Symbols". For source files and diagrams, keep the shorter namespace/prefix `cubeocc`.

3. Optical frame and link protocol

3.1 8x8 cell accounting

For an 8x8 frame there are 64 visible cells. Four corner cells are permanent white geometric markers during normal DATA frames. One additional fixed cell is the frame-toggle/clock indicator. That leaves 59 cells carrying coded information.

Item	Cells	Information
Corner markers	4	Geometry / ROI lock. White during normal DATA frames.
Frame toggle / CLK	1	Changes state for every new transmitted DATA frame so repeated camera captures can be rejected.
RGB coded cells	59	Each one represents 3 bits using the 8 RGB cube corners.
Total	64	8x8 optical grid.

Raw coded capacity per DATA frame = 59 cells x 3 bits/cell = 177 bits. At F accepted DATA frames/s, raw coded rate = 177 x F bit/s. For example, 30 accepted DATA frames/s is 5.31 kbit/s raw, while 60 accepted DATA frames/s is 10.62 kbit/s raw. Useful payload goodput is lower after headers, CRC, ECC, repeated control frames, and dropped frames.

8x8 optical frame concept

M	CLK*	D	D	D	D	D	M
D	D	D	D	D	D	D	D
D	D	D	D	D	D	D	D
D	D	D	D	D	D	D	D
D	D	D	D	D	D	D	D
D	D	D	D	D	D	D	D
D	D	D	D	D	D	D	D
M	D	D	D	D	D	D	M

M = fixed white corner marker; D = RGB data/ECC symbol; CLK* = example only - choose one fixed non-corner location

Conceptual 8x8 layout. The CLK location shown is only an example; choose one fixed non-corner cell and make it a protocol constant.

3.2 RGB symbol mapping

The simplest mapping is also a good one: let the three data bits directly control the R, G, and B channels. Adjacent RGB-cube vertices differ by one channel/bit, so a one-channel classification error causes a one-bit symbol error rather than an arbitrary 3-bit change.

Bits	Nominal symbol	RGB channel state
000	Black	R=0, G=0, B=0
001	Blue	0,0,1
010	Green	0,1,0
011	Cyan	0,1,1
100	Red	1,0,0
101	Magenta	1,0,1

110	Yellow	1,1,0
111	White	1,1,1

3.3 START / DATA / STOP state machine

Your optical preamble idea is useful because it allows the receiver to operate without a wired start signal. The physical button belongs to the transmitter side; in simulation the testbench asserts it to begin generating optical frames. The receiver starts only after recognizing the optical START pattern.

State / frame	Proposed behavior	Recommended robustness rule
IDLE	Receiver searches for START.	Do not decode payload while idle. Keep tracker running.
START	All 59 data cells white; four markers white; CLK held high/white.	Require 2-3 consecutive START frames in physical tests. One frame is acceptable for early simulation.
DATA	Four markers white. CLK toggles on every newly transmitted optical frame. 59 cells carry coded stream.	Accept only when marker geometry is valid and CLK indicates a new frame. Include an ECC-protected sequence number in the coded stream.
STOP	59 data cells black; CLK low/black. User option: lower-right marker also off.	Require 2-3 matching STOP frames. If lower-right marker is off, validate with the last good homography and the other three markers.
DONE	Flush decoder, verify CRC/length, emit file/bitstream and metrics.	Return to IDLE after completion.

Important clock-cell limitation: a 1-bit toggle tells you that a captured frame differs from the previous accepted frame, but it cannot uniquely count missed frames. If the camera misses two transmitted frames, the toggle can return to the same state. A small modulo frame sequence number (for example 4-8 bits) inside the ECC-protected data stream solves that measurement and reassembly problem.

The proposed STOP frame intentionally violates the "four markers are always white" rule by turning off the lower-right marker. That is workable only if STOP recognition uses the geometry from the previous DATA frame or a 3-marker fallback. A cleaner alternative is to keep all four markers white and use a reserved STOP signature in the data cells. Keep both variants in the code as a compile-time protocol option.

3.4 Framing fields inside the 177-bit stream

The 59 RGB cells can still be described as "payload + ECC" even if a few decoded bits are link metadata. A practical packet stream before ECC is:

Field	Typical size	Purpose
Version / mode	2-4 bits	Protocol version, grid mode, or ECC profile.
Frame sequence	4-8 bits per optical frame	Detect missed / repeated frames and order multi-frame packets.
Payload length	16-32 bits per transfer	Know when the reconstructed file/text is complete.
Payload	Variable	Text/file bytes.
CRC	16 or 32 bits per transfer/packet	Detect residual errors after ECC.

Recommended implementation: form a continuous packet bitstream, append CRC, then ECC-encode it, then slice the coded stream into 177-bit optical DATA frames. Do not force every ECC codeword to start and end inside one optical frame unless you need that simplicity; continuous packing wastes fewer bits.

4. ECC options and expected payload efficiency

For the first working version, use one simple Hamming code. Then compare one medium and one stronger BCH profile. Keep the ECC interface generic so only the encoder/decoder block changes. The table below shows one-frame packing examples only; continuous packing across optical frames is more efficient.

ECC candidate	Correction capability	If packed independently into 177 bits	Approx. code rate	Use
None	None	177 uncoded bits	1.000	Baseline BER / classifier debugging.
Hamming(7,4)	Correct 1 bit per 7-bit codeword	25 codewords = 100 data bits, 2 coded bits unused	0.571	Very simple MVP ECC.
Hamming(15,11)	Correct 1 bit per 15-bit codeword	11 codewords = 121 data bits, 12 coded bits unused	0.733	Good light ECC comparison.
Extended Hamming(8,4)	SECDED: correct 1, detect 2	22 codewords = 88 data bits, 1 coded bit unused	0.500	Useful when double-error detection matters.
BCH(31,21), t=2	Correct up to 2 bit errors/codeword	5 codewords = 105 data bits, 22 coded bits unused	0.677	Medium option.
BCH(31,16), t=3	Correct up to 3 bit errors/codeword	5 codewords = 80 data bits, 22 coded bits unused	0.516	Stronger option.

For BCH, confirm the generator polynomial and exact decoder implementation used by your chosen library/tool. A hardware BCH decoder is significantly more work than Hamming, so it should not block the first integrated demo.

4.1 Metrics to log

Metric	Definition / reason
Symbol error rate (SER)	Wrong RGB symbol cells / total decoded data cells. Shows the vision/classifier quality directly.
Raw coded BER	Bit differences between the 177 recovered coded bits and transmitted coded bits before ECC.
Post-ECC BER	Bit differences after ECC decode. This is the main communication-quality metric.
Frame error rate (FER)	Optical DATA frames that are dropped, uncorrectable, or fail frame-level checks.
Packet/file success	CRC pass/fail and exact reconstruction.
Raw coded throughput	Accepted DATA frames/s x coded bits/frame.
Goodput	Correct payload bits/s after ECC, headers, control frames, and retransmission/duplicates if any.
Latency	Cycles or time from frame start to decoded bytes.
ECC stats	Corrected codewords/bits and uncorrectable events.
Tracking stats	Marker lock rate, ROI confidence, homography error, duplicated/missed frame count.

5. Architecture review: problems and fixes

5.1 Full-frame storage in FFs/latches is the largest resource risk

An FPGA may contain many flip-flops, but using them as a megapixel image store is usually the wrong architecture. One 640x480 RGB888 frame is 640 x 480 x 24 = 7,372,800 bits. Three frames would require more than 22 million bit registers before any processing logic. Even a 128x128 RGB ROI is 393,216 bits per frame; three such FF-backed frames exceed one million bit registers.

Image / ROI	Bits per RGB888 frame	Approx. 36-kbit BRAM blocks/frame	Three-frame storage
640x480	7,372,800	200	22,118,400 bits

320x240	1,843,200	50	5,529,600 bits
256x256	1,572,864	43	4,718,592 bits
128x128	393,216	11	1,179,648 bits

Recommended memory hierarchy: stream pixels whenever possible; use line buffers for local filters; store only the cropped ROI if perspective sampling requires random access; use two or three BRAM/URAM banks as ping-pong/PIPO buffers; keep only descriptors and small neighborhoods in FFs. If the eventual board has external DDR, it can hold full frames, but the one-month simulation should not require DDR.

5.2 Three complete processing workers are optional, not the baseline

Your scheduler idea is valid: frame 1 can occupy worker 1, frame 2 worker 2, frame 3 worker 3, and a free-worker bitmap can dispatch later frames. The issue is resource multiplication: denoise, warp, color averaging, and classification logic may be duplicated three times, and each worker may need its own image bank.

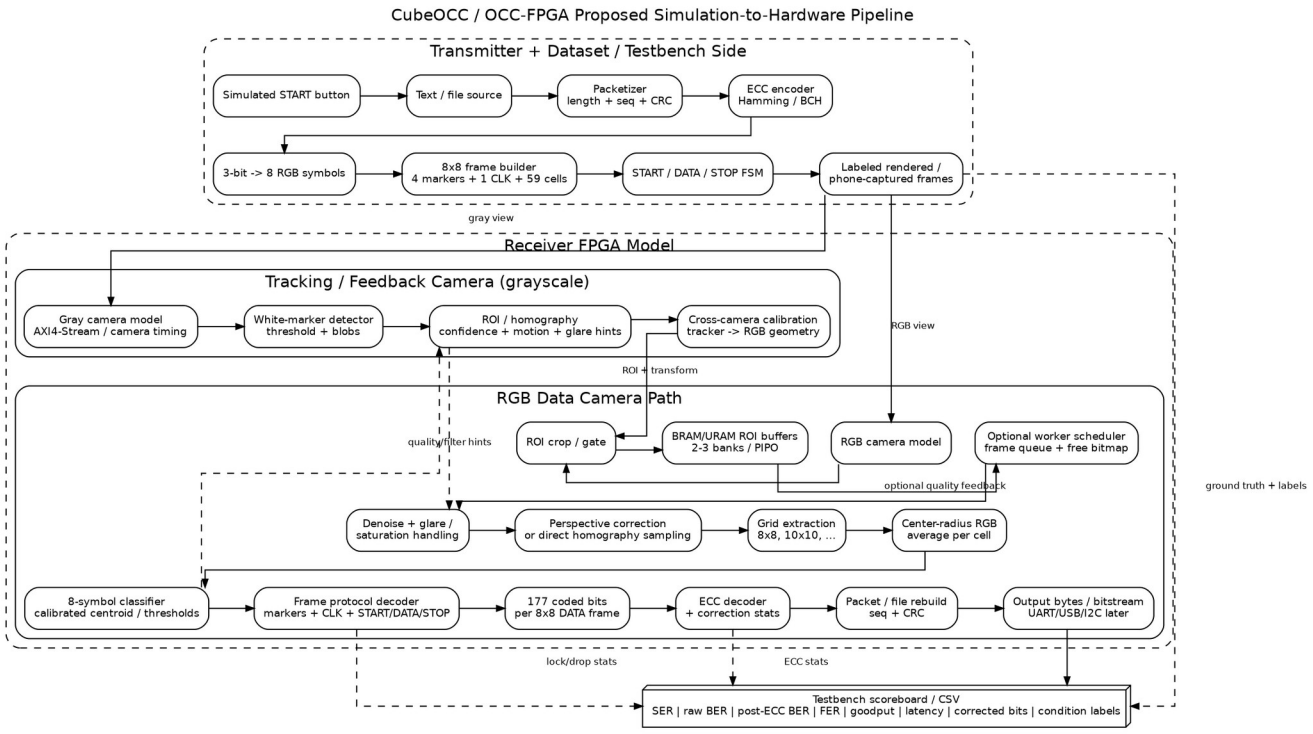
Baseline recommendation: make one image pipeline internally concurrent with HLS DATAFLOW and streaming FIFOs. If synthesis shows that one pipeline cannot meet the desired accepted frame rate and enough resources remain, then duplicate only the bottleneck stage or instantiate 2-3 workers. This gives measurable justification for the scheduler rather than making it mandatory from day one.

5.3 Dual-camera mapping requires calibration, not only a fixed offset

A rigid mounting does not guarantee a constant pixel offset between two cameras. Different focal lengths, lens distortion, image sizes, optical centers, and perspective can create scale and projective differences. Represent the mapping as calibration parameters, preferably a homography for the planar LED panel after lens effects are handled or ignored in the MVP.

Simulation without two cameras: use one RGB source image as the "data camera" view, then synthetically generate the grayscale tracker view by applying a known downscale, crop, offset, blur/noise, and projective transform. The testbench then knows the true cross-camera transform, so you can measure ROI mapping error before hardware exists.

6. Proposed end-to-end pipeline



Recommended pipeline. The grayscale camera continuously estimates geometry and quality hints; the RGB data path performs decoding. Dashed paths are feedback/metrics rather than the main pixel stream.

Key feedback path: Gray tracker -> marker geometry -> cross-camera mapping -> RGB ROI/warp parameters. Optional quality flags select filter strength. Decoder confidence can also be fed back to adaptive color thresholds, but this should be a stretch feature.

6.1 Tracker / grayscale camera path

1. Receive a camera-like grayscale pixel stream with frame/line validity.
2. Threshold bright pixels to find white marker candidates. In the MVP, use a fixed/adaptive threshold plus area constraints rather than a complex feature detector.
3. Find connected components or bounding boxes, reject blobs outside expected size/shape, and choose four candidates that form a plausible quadrilateral near the image corners.
4. Order markers as top-left, top-right, bottom-left, bottom-right. Compute ROI bounds and a 3x3 projective transform/homography to a canonical rectangle.
5. Convert tracker coordinates to the RGB-camera coordinate system using fixed calibration parameters. Output ROI, transform, marker confidence, motion estimate, brightness/saturation hints, and a valid flag.

Implementation option	Complexity	Recommendation
Fixed brightness threshold + four largest valid blobs	Low	Start here. Works well with intentionally bright white corner markers.
Adaptive/Otsu threshold + geometry filtering	Medium	Good second step for changing ambient brightness.
Template/corner feature detector	Higher	Probably unnecessary for the first month unless markers are unreliable.

6.2 RGB camera / ROI input

The higher-quality RGB camera is the data path. In simulation, the testbench should not simply call the HLS function with a completed image array if the goal is future hardware realism. Feed pixels as a timed stream with start-of-frame, end-of-line/last, valid/ready, and optional blanking/stalls.

Option	Pros	Cons	Use
AXI4-Stream-like pixel interface	Maps naturally to HLS streams, FIFOs, ready/valid backpressure, and downstream IP.	Not identical to every raw camera sensor interface.	Recommended simulation abstraction.
DVP-style pclk/href/vsync model	Looks like a simple parallel camera sensor.	More RTL wrapper work; less convenient for HLS dataflow.	Use later if target sensor needs it.
Array/image argument in C testbench	Fastest algorithm debug.	Does not exercise streaming/backpressure behavior.	Keep only as a unit-test/golden path.

7. Image-processing units and options

7.1 ROI buffering and scheduler

If the RGB ROI must be randomly accessed for perspective transformation, write it into BRAM/URAM. Use a small bank descriptor FIFO: `{bank_id, frame_seq, roi_width, roi_height, transform, quality_flags}`. A bank changes states FREE -> FILLING -> READY -> PROCESSING -> FREE.

Architecture	Memory	Parallelism	Recommendation
Single streaming worker + line buffers	Lowest	Pipeline stages overlap within one frame.	MVP.
Double-buffered ROI (ping-pong/PIPO)	Moderate	Capture next ROI while processing current ROI.	Recommended if warp needs buffering.
Three ROI banks + three workers	High	Up to three frames processed concurrently.	Stretch after resource/latency measurements.

7.2 Denoising

Method	Hardware cost	Behavior	Recommendation
3x3 median	Low-medium	Strong against salt/pepper and isolated sensor noise; needs line buffers.	Good MVP option.
3x3 box / Gaussian-like filter	Low	Reduces random noise but blurs cell edges.	Useful before center sampling; keep kernel small.
Bilateral filter	High	Preserves edges while smoothing.	Avoid in one-month FPGA MVP.
No full-image denoise; average only center patches	Very low	Your later cell averaging already rejects much noise.	Often best first baseline.

7.3 Sun glare / brightness variation

Do not plan on "removing sun glare" perfectly. Saturated pixels have already lost information. A more realistic goal is to detect saturation, avoid edge/highlight regions when sampling, normalize channel values, and flag low-confidence frames.

Technique	What it helps	Cost / note
Saturation mask	Ignore pixels close to 255 in one or more channels when computing cell averages.	Very cheap.
Center-radius sampling	Avoid cell borders, reflections, and marker bleed.	Already in your design; keep it.
Normalized chromaticity r=R/(R+G+B), etc.	Makes color decision less sensitive to overall brightness.	Requires reciprocal/division; fixed-point approximation or LUT can be used.
Per-frame/channel gain from known white markers	Uses markers as a brightness/white-balance reference.	Very attractive for this project.
Local tone mapping / CLAHE	Can improve difficult scenes.	Too expensive for baseline; evaluate only if simpler normalization fails.

7.4 Perspective correction

Option	Description	Pros	Cons
Full ROI warp	Inverse-map the quadrilateral into a rectangular image using nearest-neighbor or bilinear interpolation.	Conceptually simple; easy to compare with OpenCV.	Consumes buffering/bandwidth; full image work may be unnecessary.
Direct grid sampling	Use the homography to predict each cell center/patch in the original ROI and accumulate only those pixels.	Can avoid producing a full rectified frame; lower work for small grids.	Random/irregular memory access and more careful coordinate logic.
Affine-only correction	Use 2x3 transform when viewpoint is nearly frontal.	Cheaper.	Fails under significant perspective.

Recommended progression: implement full nearest-neighbor perspective warp first for correctness. If resource use is too high, switch to direct homography sampling of cell centers/patches. The current AMD Vitis Vision library also provides a streaming warpTransform primitive that buffers input rows in BRAM/URAM; it can be a reference or implementation option if your installed tool version supports it.

8. Grid extraction and 8-symbol classifier

8.1 Grid extraction

After perspective correction, the panel is a canonical rectangle. For an N x N grid, compute integer/fixed-point cell boundaries from ROI width and height. Do not classify the whole cell: sample only a disk or square patch around the center, with radius expressed as a fraction of cell size (for example 20-30% of half-width). This avoids grid lines, marker edges, and perspective interpolation artifacts.

Parameter	Suggested configurable value
Grid size	8x8 first; later 10x10 or parameterized N.
Center patch radius	Start around 20-30% of cell half-size; sweep in simulation.
Minimum valid pixels	Reject a cell if too many samples are saturated/out of ROI.
Accumulator width	Size for worst-case sample count x 255; use integer/fixed point.

8.2 Classifier options

Classifier	How it works	Hardware cost	Recommendation
Three independent thresholds	Decide R/G/B bit by comparing each average channel against a threshold.	Very low	Best baseline; natural 3-bit mapping.
Nearest of 8 RGB centroids	Store calibrated (R,G,B) centroids for each symbol and choose minimum squared distance.	Low-medium	Recommended robust mode.
Normalized/chromaticity centroid	Classify using normalized channel ratios and brightness features.	Medium	Useful under lighting variation.
Small ML classifier	Learn decision boundaries from labeled samples.	Higher + harder to verify	Not needed for this project.

A very useful calibration trick is to use the START all-white frame and known white markers to estimate white level, while STOP/black cells estimate black level. If you later add a short calibration preamble containing all eight colors, you can estimate all eight symbol centroids automatically for each lighting condition.

8.3 Confidence score

For each cell, keep not only the chosen 3-bit symbol but also a confidence measure, such as the distance to the nearest centroid versus the second-nearest centroid. Low-confidence cells can be counted for diagnostics and, in a later experiment, used as erasures for a decoder that supports erasure information.

9. HLS / RTL integration strategy

Use a clean stream-oriented top level so the simulation can later be connected to a physical camera wrapper. In modern AMD Vitis HLS, AXI4-Stream uses TVALID/TREADY handshaking and can carry TLAST/TUSER side information; DATAFLOW lets producer/consumer processes overlap through FIFOs/PIPO channels. Similar stream/dataflow concepts exist in older Vivado HLS flows, although exact APIs and generated interfaces depend on the version.

Module	Suggested interface	State kept on chip
tracker_top	gray pixel stream -> feedback stream/registers	line buffers, blob stats, marker coordinates, transform.
rgb_receiver_top	RGB pixel stream + feedback -> decoded byte/bit stream	ROI banks/line buffers, accumulators, protocol/ECC state.
frame_protocol	symbol stream -> coded-bit stream	CLK history, receiver FSM, frame sequence, counters.
ecc_decode	coded-bit stream -> decoded-bit stream	codeword shift registers/syndrome state.
scoreboard (TB only)	decoded output + ground truth + labels	host memory/files; not synthesized.

Do not synthesize the dataset reader, CSV writer, or filesystem access. Those are testbench/host functions. The synthesizable design should see only pixel/control streams and produce decoded bits/bytes plus status counters.

10. Testbench that behaves like real cameras

10.1 Dataset manifest

Keep every captured/synthetic scene in a manifest so the scoreboard can group results by condition. A CSV row can contain:

Field	Example / meaning
scene_id	unique integer/string
rgb_image	path to RGB data-camera frame
gray_image	path or generated tracker view
ground_truth_frame_seq	expected optical frame number
frame_type	START / DATA / STOP
distance_cm	capture distance
yaw_deg / pitch_deg	viewing angle
ambient	indoor / daylight / glare
blur_level	label or numeric estimate
grid_size	8
ecc_mode	none / hamming15_11 / bch...
expected_coded_bits	optional golden 177-bit frame
expected_payload	transfer/file identifier

10.2 Camera stream model

6. Read the image file and metadata in the testbench.
7. For the grayscale tracker stream, use a separate image or generate one from the RGB image with downsampling, blur/noise, and a known spatial transform.
8. Emit pixels line by line with realistic frame/line control. Optionally insert random ready/valid stalls or blanking periods.
9. Drive both camera streams with their own frame cadence. The tracker can run faster/lower quality; the RGB path can run slower/higher quality.
10. When the simulated transmitter button is asserted, begin feeding START frames, then DATA frames, then STOP frames. The receiver is not directly told that the button was pressed.
11. Compare decoded output with ground truth and append one result row per scene/transfer to CSV/TXT.

For hardware realism, also test camera/display frame-rate mismatch: repeat the same optical frame multiple camera captures, skip some optical frames, and add timing jitter. Verify that CLK duplicate suppression and sequence-number loss detection behave correctly.

10.3 Golden reference model

Maintain a software reference (Python/OpenCV or plain C++) for marker geometry, perspective warp, grid sampling, symbol classification, packet/ECC operations, and BER. The HLS blocks should be compared against this golden model stage by stage. This makes fixed-point errors and HLS-specific bugs much easier to locate.

11. Suggested GitHub structure

A repository layout that keeps synthesizable code, testbench code, data, and evaluation separate:

```
cubeocc-fpga/
  README.md
  docs/
    protocol.md
    architecture.md
  hls/
    include/
      cubeocc_types.hpp
      protocol_cfg.hpp
    src/
      tracker.cpp
      roi_buffer.cpp
      preprocess.cpp
      warp.cpp
      grid_sampler.cpp
      symbol_classifier.cpp
      frame_protocol.cpp
      ecc_hamming.cpp
      receiver_top.cpp
    tb/
      camera_model.cpp
      tb_receiver.cpp
  scripts/
    build_manifest.py
    generate_optical_frames.py
    evaluate_results.py
  data/
    manifest.csv
    raw/
    generated/
  results/
    .gitkeep
  tests/
    golden_vectors/
```

12. One-month plan for two people

Week	Person A - vision / HLS	Person B - protocol / testbench	Integration milestone
1	Define pixel types; implement marker threshold/blob baseline; create Python geometry reference.	Freeze 8x8 protocol; symbol mapper; START/DATA/STOP generator; dataset manifest + camera stream model.	One known optical frame can be streamed into HLS and compared with ground truth.
2	ROI mapping, perspective warp or direct sampling, grid center averaging, simple RGB thresholds.	Frame FSM, CLK duplicate detection, sequence numbers, no-ECC and Hamming encoder/decoder, scoreboard CSV.	End-to-end text bits pass through clean synthetic images.
3	Add denoise/normalization options, calibrated centroids, tracker confidence, synthetic second-camera transform.	Add CRC, ECC comparison framework, dropped/repeated-frame tests, condition labels and plots/scripts.	Phone-captured / degraded image dataset decodes with measured BER.
4	Profile HLS latency/resources; optimize buffers/dataflow; optional 2nd worker or direct homography sampler.	Run experiment matrix, BCH stretch option, final metrics tables, README/demo scripts.	Reproducible demo: button -> optical frame sequence -> camera-like simulation -> reconstructed text + CSV metrics.

Scope gate at end of Week 2: if the complete clean-image 8x8 path is not decoding text yet, postpone BCH, 10x10, complex glare correction, and three-worker scheduling. A correct measured baseline is more valuable than many half-integrated features.

13. Verification checklist

- Protocol: START recognized only in IDLE; DATA accepted only on new CLK state; STOP terminates and flushes.
- Frame sequence: repeated frames ignored; skipped frames detected; order checked.
- Marker detector: all four corners ordered correctly under rotation/perspective within target range.
- Cross-camera mapping: tracker ROI mapped to RGB view within a measured pixel error bound.
- Warp/grid: canonical cell centers match golden model within tolerance.
- Classifier: confusion matrix produced for all 8 symbols under each lighting condition.
- ECC: inject 0/1/2/... bit errors and verify corrected/uncorrectable behavior for every selected code.
- Bitstream reconstruction: exact text/file output and CRC pass on successful transfers.
- Metrics: SER, raw BER, post-ECC BER, FER, goodput, latency, and condition labels all written to CSV.
- HLS: C simulation, C synthesis, RTL co-simulation, and dataflow/channel-depth inspection performed before optimization.

14. Final recommended architecture choices

Block	MVP choice	Next option / stretch
Project name	CubeOCC / cubeocc-fpga	ChromaOCC if you want a more brand-like name.
Grid	8x8	10x10 parameterization after stable 8x8.
Markers	4 white corners	Add marker IDs/patterns only if ordering becomes ambiguous.
New-frame indicator	1 toggling CLK cell	Keep + ECC-protected 4-8 bit sequence counter.
Frame control	White START, black STOP, repeated 2-3x physically	Keep lower-right marker white if possible; otherwise 3-marker STOP fallback.
Storage	Streaming + line buffers + 2 ROI BRAM banks	3 banks/workers only after resource profiling.
Denoise	Center averaging or 3x3 median	Adaptive filter based on tracker quality flag.
Glare handling	Saturation mask + marker-based gain + normalized classification	Local tone mapping only if needed.
Perspective	Nearest-neighbor homography warp	Bilinear or direct grid sampling.
Classifier	3 channel thresholds	Nearest calibrated 8-color centroid.
ECC	Hamming(15,11) or (7,4)	BCH medium/strong comparison.
Output	Decoded bit/byte stream + status	UART/USB bridge/I2C wrapper on physical board.

15. References and current AMD implementation notes

These references are not requirements for the project; they support the recommended stream/dataflow architecture and show that the proposed operations map naturally to current AMD FPGA image-processing flows.

[1] AMD, Vitis High-Level Synthesis User Guide UG1399, 2026.1 (release 2026-06-23): "How AXI4-Stream Works", "Tasks and Dataflow", "Stream", and "Dataflow Viewer". These sections document ready/valid streaming, hls::stream/FIFO behavior, and DATAFLOW process overlap.

[2] AMD, Vitis Vision Library, 2026.1: "Warp Transform". The library supports affine/perspective transforms with nearest-neighbor or bilinear interpolation and describes buffering input rows in BRAM/URAM for streaming geometric transforms.

[3] AMD, Vitis Vision Library, 2026.1: "Resolution Conversion (Resize)" and current Vision API index. Useful if the tracker path needs explicit downsampling or other standard vision primitives.

[4] N. T. Le, M. A. Hossain, Y. M. Jang, "A survey of design and implementation for optical camera communication," Signal Processing: Image Communication, 2017. The literature uses OCC to mean Optical Camera Communication and discusses camera-based optical links, synchronization, perspective, and image-sensor constraints.

[5] N. Saeed et al., "Optical camera communications: Survey, use cases, challenges, and future trends," Physical Communication, 2019. Background on OCC modulation, coding, synchronization, and image-processing challenges.

Tool-version note: if your course/lab specifically uses legacy Vivado HLS rather than current Vitis HLS, keep the architecture but adapt pragma/interface syntax to the installed version. The report intentionally describes interfaces conceptually so the design remains portable.